

Travaux Pratique N°4

Interface Graphique pour une API REST Django

Objectif

Ce TP est la suite directe du **TP3** où vous avez exposé les modèles **Auteur**, **Livre** et **Emprunt** sous forme d'API REST. Vous allez maintenant créer une **interface graphique web** permettant d'interagir avec cette API via des appels REST.

Travail à réaliser

1. Préparation de la structure du projet

Avant de créer l'application avec la commande :

```
python manage.py startapp UI_bibliotheque
```

il est recommandé de créer manuellement deux dossiers à la racine du projet :

- `templates/` : contient les fichiers HTML.
- `static/` : contient les fichiers CSS, JavaScript et images.

Ces dossiers permettront de centraliser les fichiers réutilisables et d'éviter la duplication dans chaque application.

2. Configuration dans `settings.py`

Le fichier `settings.py` centralise la configuration du projet Django. Il faut y préciser l'emplacement des dossiers `templates` et `static` pour que le framework puisse les détecter.

```

1 import os
2 from pathlib import Path
3
4 BASE_DIR = Path(__file__).resolve().parent.parent
5
6 TEMPLATES = [
7     {
8         'BACKEND': 'django.template.backends.django.
          DjangoTemplates',
9         'DIRS': [BASE_DIR / 'UI_Bibliotheque' / '
            templates'],
10        'APP_DIRS': True,
11        'OPTIONS': {'context_processors': [...]}},
12    },
13 ]

```

```

1 STATIC_URL = '/static/'
2 STATICFILES_DIRS = [os.path.join(BASE_DIR, 'static')]
3 STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')

```

3. Création du dossier des templates d'application

- Créez un dossier `templates/bib/` dans l'application TP3.
- Configurez ce dossier dans le fichier `settings.py` si nécessaire.
- Créez un fichier `base.html` qui contiendra la structure HTML commune à toutes les pages :
 - En-tête (`<header>`)
 - Barre de navigation
 - Contenu principal
 - Pied de page

2. Interface des auteurs

- Créez un template `auteurs.html` affichant la liste des auteurs récupérés via l'API `/api/auteurs/`.
- Ajoutez un bouton permettant d'ajouter un nouvel auteur via un formulaire HTML.
- Lors de la soumission, envoyez les données à l'API avec une requête `POST`.
- Ajoutez la possibilité de supprimer un auteur avec une requête `DELETE`.

3. Interface des livres

- Créez un template `livres.html` affichant la liste des livres.
- Ajoutez un bouton `Ajouter un livre` permettant d'envoyer une requête `POST` vers `/api/livres/`.
- Implémentez la suppression d'un livre avec un bouton `Supprimer`.
- Ajoutez un lien pour modifier un livre (`PUT`).

4. Interface des emprunts

- Créez un template `emprunts.html` affichant tous les emprunts.
- Affichez les informations : `livre`, `auteur`, `date d'emprunt`, `date de retour`.
- Ajoutez un formulaire permettant d'enregistrer un nouvel emprunt (via `POST`).
- Implémentez la possibilité de marquer un emprunt comme retourné (via `PUT`).

5. Navigation entre les pages

- Ajoutez une barre de navigation dans `base.html` avec des liens vers :
 - `/auteurs/`

- `/livres/`
- `/emprunts/`
- Créez les vues Django correspondantes pour chaque page.
- Testez les transitions entre les pages.
-

6. Tests et validation

- Testez la création, la modification et la suppression d'un auteur, livre et emprunt via l'interface.
- Vérifiez les mises à jour dans la base de données via l'interface d'administration.
- Assurez-vous que toutes les pages communiquent uniquement avec l'API (aucune requête directe à la base).